

Practical 3

ID: 17CE049

Subject : CE473 Machine Learning

Aim : Multivariate Linear Regression with KPI

Task1: Implement Multivariate Linear Regression over Hiring.csv Dataset

You need to build a machine learning model for HR department that can help them decide salaries for future candidates. Using this predict salaries for following candidates,

2 yr experience, 9 test score, 6 interview score 12 yr experience, 10 test score, 10 interview score

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [3]:

```
df = pd.read_csv('hiring.csv')
df.head()
```

Out[3]:

	experience	test_score(out of 10)	interview_score(out of 10)	salary(\$)
0	NaN	8.0	9	50000
1	NaN	8.0	6	45000
2	five	6.0	7	60000
3	two	10.0	10	65000
4	seven	9.0	6	70000

In [4]:

```
df
```

Out[4]:

	experience	test_score(out of 10)	interview_score(out of 10)	salary(\$)
0	NaN	8.0	9	50000
1	NaN	8.0	6	45000
2	five	6.0	7	60000
3	two	10.0	10	65000
4	seven	9.0	6	70000
5	three	7.0	10	62000
6	ten	NaN	7	72000
7	eleven	7.0	8	80000

In [5]:

```
df.experience
```

Out[5]:

```
0      NaN
1      NaN
2     five
3      two
4     seven
5     three
6      ten
7    eleven
Name: experience, dtype: object
```

In [6]:

```
df.experience = df.experience.fillna('zero')
df.experience
```

Out[6]:

```
0      zero
1      zero
2     five
3      two
4     seven
5     three
6      ten
7    eleven
Name: experience, dtype: object
```

In [20]:

```
from word2number import w2n
df.experience = df.experience.apply(w2n.word_to_num)
```

In [21]:

```
df.experience
```

Out[21]:

```
0    0
1    0
2    5
3    2
4    7
5    3
6   10
7   11
```

Name: experience, dtype: int64

In [22]:

```
import math
median_test = math.floor(df['test_score(out of 10)'].median())
```

In [23]:

```
median_test
```

Out[23]:

8

In [24]:

```
df['test_score(out of 10)'] = df['test_score(out of 10)'].fillna(median_test)
df
```

Out[24]:

	experience	test_score(out of 10)	interview_score(out of 10)	salary(\$)
0	0	8.0	9	50000
1	0	8.0	6	45000
2	5	6.0	7	60000
3	2	10.0	10	65000
4	7	9.0	6	70000
5	3	7.0	10	62000
6	10	8.0	7	72000
7	11	7.0	8	80000

In [25]:

```
X = df.iloc[:, :3].values  
y = df.iloc[:, -1].values
```

In [26]:

```
X
```

Out[26]:

```
array([[ 0.,  8.,  9.],  
       [ 0.,  8.,  6.],  
       [ 5.,  6.,  7.],  
       [ 2., 10., 10.],  
       [ 7.,  9.,  6.],  
       [ 3.,  7., 10.],  
       [10.,  8.,  7.],  
       [11.,  7.,  8.]])
```

In [27]:

```
y
```

Out[27]:

```
array([50000, 45000, 60000, 65000, 70000, 62000, 72000, 80000],  
      dtype=int64)
```

In [28]:

```
from sklearn.linear_model import LinearRegression  
regressor = LinearRegression()  
regressor.fit(X, y)
```

Out[28]:

```
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)
```

In [29]:

```
regressor.coef_
```

Out[29]:

```
array([2812.95487627, 1845.70596798, 2205.24017467])
```

In [30]:

```
regressor.intercept_
```

Out[30]:

```
17737.263464337695
```

In [31]:

```
regressor.score(X, y)
```

Out[31]:

0.961678000257796

In [32]:

```
regressor.predict([[5,6,7]])
```

Out[32]:

array([58312.95487627])

In [33]:

```
print("Salary of candidate having 2 yr experience, 9 test score, 6 interview score is ",regressor.predict([[2,9,6]]))
```

Salary of candidate having 2 yr experience, 9 test score, 6 interview score is [53205.96797671]

In [34]:

```
print("Salary of candidate having 12 yr experience, 10 test score, 10 interview score is ",regressor.predict([[12,10,10]]))
```

Salary of candidate having 12 yr experience, 10 test score, 10 interview score is [92002.18340611]

Task 2 UNDERSTAND AND IMPLEMENT VARIOUS REGRESSION KPIs (Key Performance Indicator)

Graduate_admission_Prediction.ipynb included in this file

In [40]:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from jupyterthemes import jtplot
jtplot.style(theme='monokai', context='notebook', ticks=True, grid=False)
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
<ipython-input-40-e2b1cbea14a4> in <module>
      3 import seaborn as sns
      4 import matplotlib.pyplot as plt
----> 5 from jupyterthemes import jtplot
      6 jtplot.style(theme='monokai', context='notebook', ticks=True, grid=False)
ModuleNotFoundError: No module named 'jupyterthemes'
```

ModuleNotFoundError: No module named 'jupyterthemes'

In []:

```
# read the csv file
admission_df=pd.read_csv('Admission_Predict.csv')
```

In [36]:

```
admission_df.head()
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-36-17381c8a7b04> in <module>
----> 1 admission_df.head()
```

NameError: name 'admission_df' is not defined

In [24]:

```
# Let's drop the serial no.
admission_df.drop('Serial No.',axis=1,inplace=True)
admission_df
```

Out[24]:

	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
0	337	118	4	4.5	4.5	9.65	1	0.92
1	324	107	4	4.0	4.5	8.87	1	0.76
2	316	104	3	3.0	3.5	8.00	1	0.72
3	322	110	3	3.5	2.5	8.67	1	0.80
4	314	103	2	2.0	3.0	8.21	0	0.65
...	...	...	...	...	...	...	...	...
495	332	108	5	4.5	4.0	9.02	1	0.87
496	337	117	5	5.0	5.0	9.87	1	0.96
497	330	120	5	4.5	5.0	9.56	1	0.93
498	312	103	4	4.0	5.0	8.43	0	0.73
499	327	113	4	4.5	4.5	9.04	0	0.84

500 rows × 8 columns

In [25]:

```
# checking the null values
admission_df.isnull().sum()
```

Out[25]:

```
GRE Score      0
TOEFL Score    0
University Rating  0
SOP            0
LOR            0
CGPA           0
Research       0
Chance of Admit  0
dtype: int64
```

In [26]:

```
# Check the dataframe information
admission_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
GRE Score      500 non-null int64
TOEFL Score    500 non-null int64
University Rating  500 non-null int64
SOP            500 non-null float64
LOR            500 non-null float64
CGPA           500 non-null float64
Research       500 non-null int64
Chance of Admit  500 non-null float64
dtypes: float64(4), int64(4)
memory usage: 31.4 KB
```

In [27]:

```
# Statistical summary of the dataframe
admission_df.describe()
```

Out[27]:

	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	C of
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500
mean	316.472000	107.192000	3.114000	3.374000	3.484000	8.576440	0.560000	0
std	11.295148	6.081868	1.143512	0.991004	0.92545	0.604813	0.496884	0
min	290.000000	92.000000	1.000000	1.000000	1.000000	6.800000	0.000000	0
25%	308.000000	103.000000	2.000000	2.500000	3.000000	8.127500	0.000000	0
50%	317.000000	107.000000	3.000000	3.500000	3.500000	8.560000	1.000000	0
75%	325.000000	112.000000	4.000000	4.000000	4.000000	9.040000	1.000000	0
max	340.000000	120.000000	5.000000	5.000000	5.000000	9.920000	1.000000	0

In [28]:

```
# Grouping by University ranking
df_university=admission_df.groupby(by='University Rating').mean()
df_university
```

Out[28]:

	GRE Score	TOEFL Score	SOP	LOR	CGPA	Research	Chance of Admit
University Rating							
1	304.911765	100.205882	1.941176	2.426471	7.798529	0.294118	0.562059
2	309.134921	103.444444	2.682540	2.956349	8.177778	0.293651	0.626111
3	315.030864	106.314815	3.308642	3.401235	8.500123	0.537037	0.702901
4	323.304762	110.961905	4.000000	3.947619	8.936667	0.780952	0.801619
5	327.890411	113.438356	4.479452	4.404110	9.278082	0.876712	0.888082

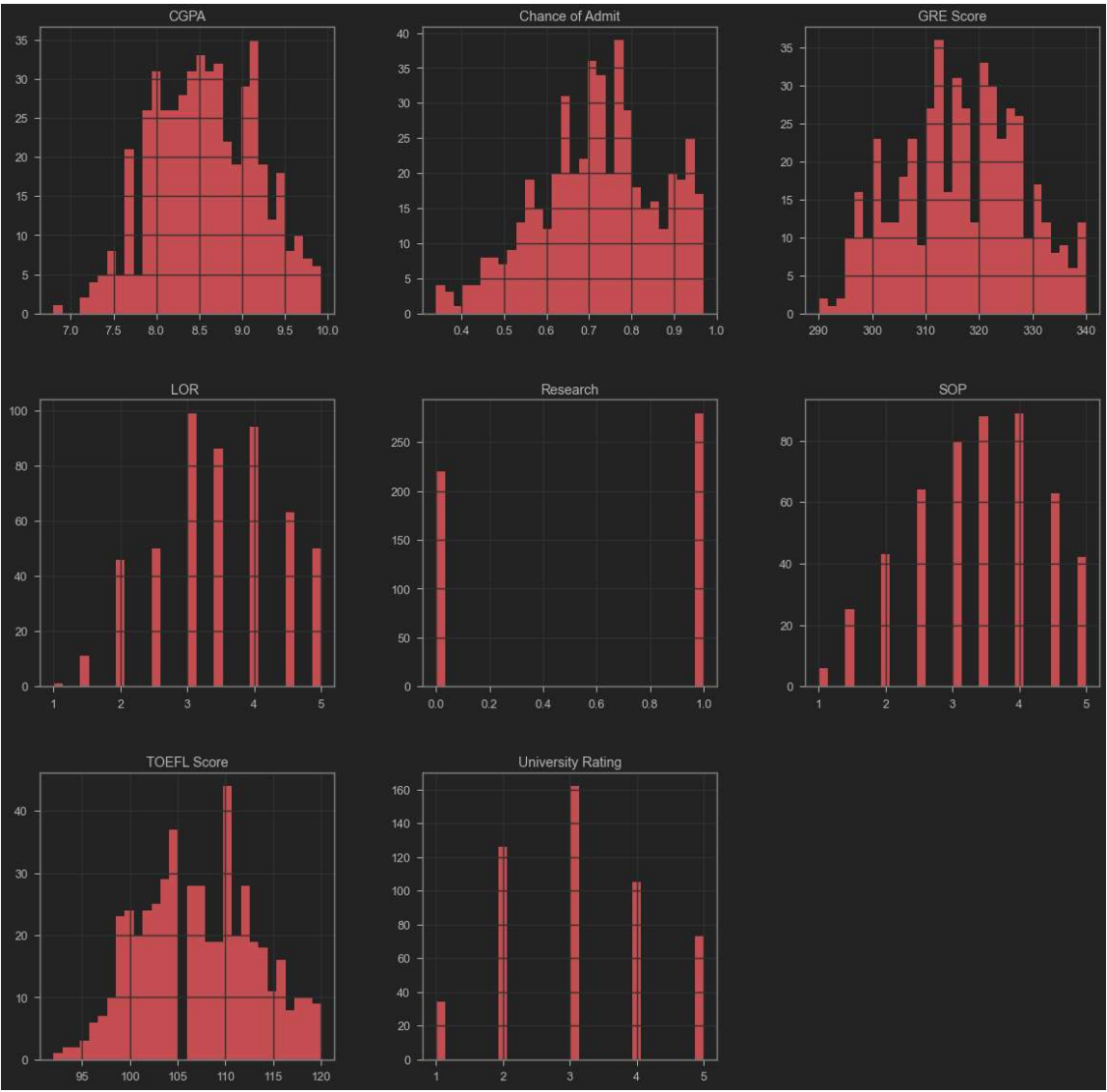
In [29]:

```
admission_df.hist(bins=30, figsize=(20,20),color='r')
```

```
C:\Users\VATSAL\Anaconda3\lib\site-packages\pandas\plotting\_matplotlib\tools.py:307: MatplotlibDeprecationWarning:
The rowNum attribute was deprecated in Matplotlib 3.2 and will be removed two minor releases later. Use ax.get_subplotspec().rowspan.start instead.
  layout[ax.rowNum, ax.colNum] = ax.get_visible()
C:\Users\VATSAL\Anaconda3\lib\site-packages\pandas\plotting\_matplotlib\tools.py:307: MatplotlibDeprecationWarning:
The colNum attribute was deprecated in Matplotlib 3.2 and will be removed two minor releases later. Use ax.get_subplotspec().colspan.start instead.
  layout[ax.rowNum, ax.colNum] = ax.get_visible()
C:\Users\VATSAL\Anaconda3\lib\site-packages\pandas\plotting\_matplotlib\tools.py:313: MatplotlibDeprecationWarning:
The rowNum attribute was deprecated in Matplotlib 3.2 and will be removed two minor releases later. Use ax.get_subplotspec().rowspan.start instead.
  if not layout[ax.rowNum + 1, ax.colNum]:
C:\Users\VATSAL\Anaconda3\lib\site-packages\pandas\plotting\_matplotlib\tools.py:313: MatplotlibDeprecationWarning:
The colNum attribute was deprecated in Matplotlib 3.2 and will be removed two minor releases later. Use ax.get_subplotspec().colspan.start instead.
  if not layout[ax.rowNum + 1, ax.colNum]:
```

Out[29]:

```
array([[<matplotlib.axes._subplots.AxesSubplot object at 0x000002882DCD7A88>,
      <matplotlib.axes._subplots.AxesSubplot object at 0x000002882DD57388>,
      <matplotlib.axes._subplots.AxesSubplot object at 0x000002882DD8A848>],
      [<matplotlib.axes._subplots.AxesSubplot object at 0x000002882DDC4448>,
      <matplotlib.axes._subplots.AxesSubplot object at 0x000002882DDFBAC8>,
      <matplotlib.axes._subplots.AxesSubplot object at 0x000002882DE32848>],
      [<matplotlib.axes._subplots.AxesSubplot object at 0x000002882DE6C708>,
      <matplotlib.axes._subplots.AxesSubplot object at 0x000002882DEA6488>,
      <matplotlib.axes._subplots.AxesSubplot object at 0x000002882DEA6688>]],
      dtype=object)
```

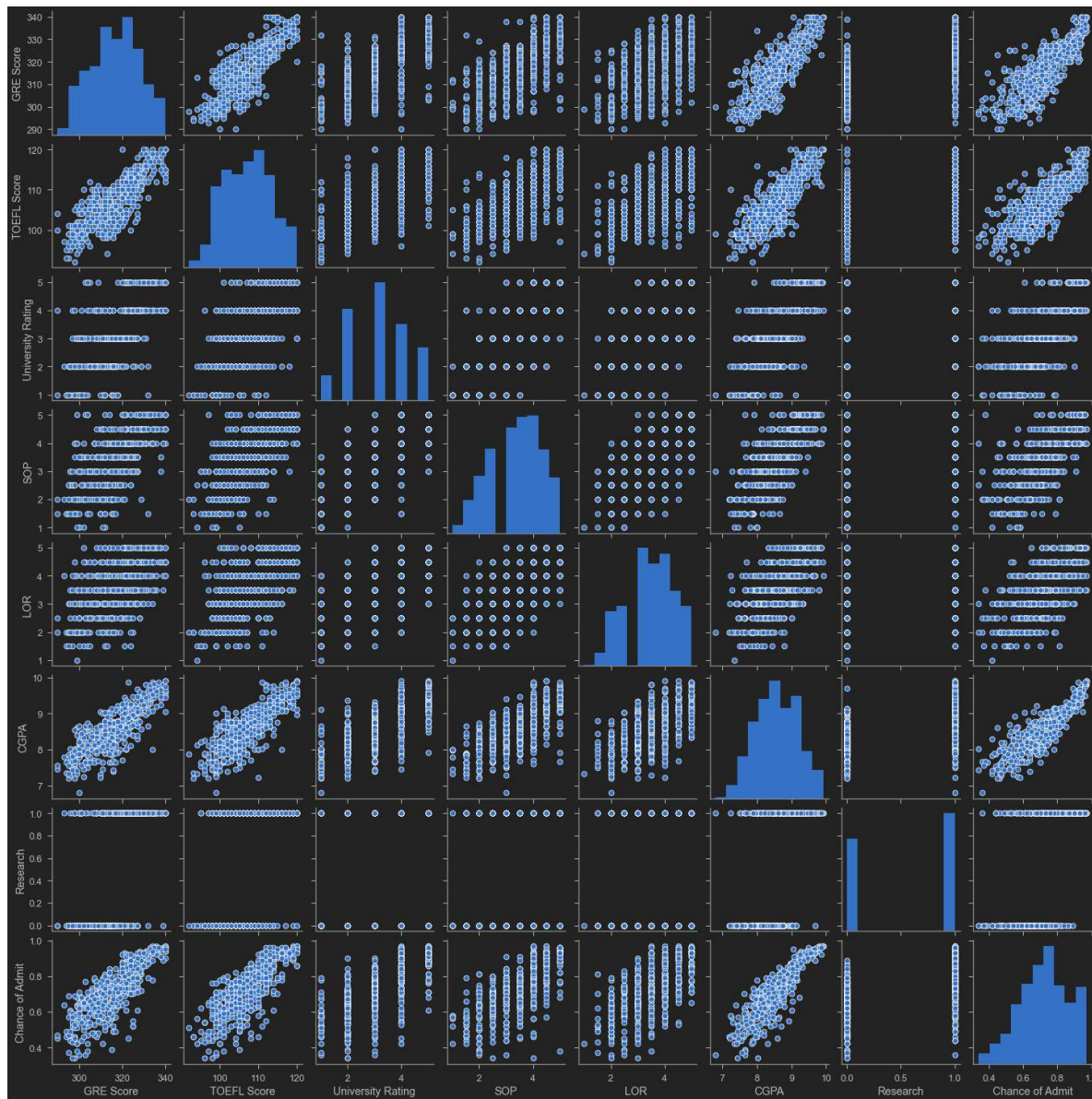


In [30]:

```
sns.pairplot(admission_df) #Pair plot for the data w.r.t. Chance of Admission
```

Out[30]:

```
<seaborn.axisgrid.PairGrid at 0x2882e110b08>
```

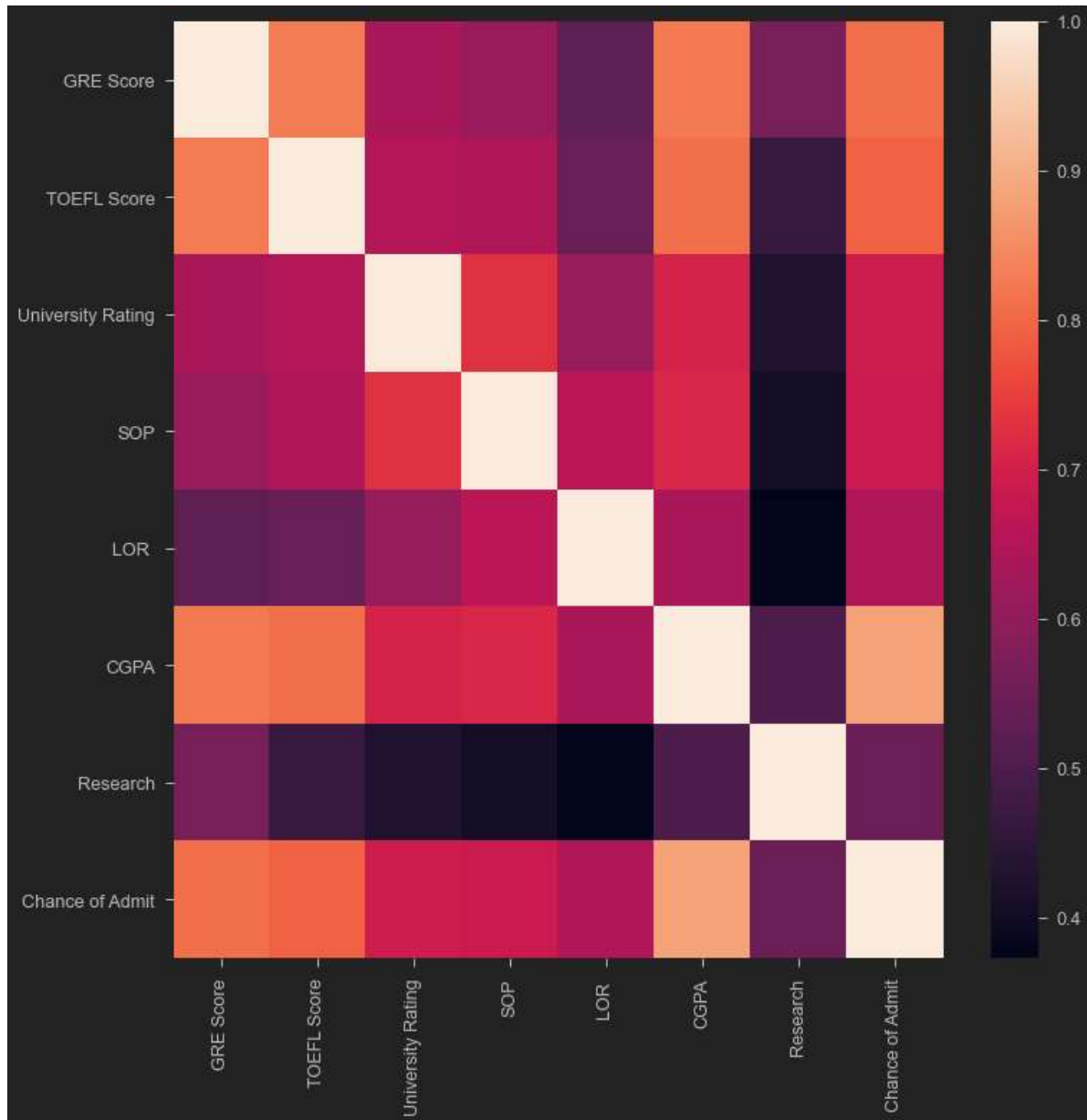


In [31]:

```
corr_matrix=admission_df.corr()  
plt.figure(figsize=(12,12))  
sns.heatmap(corr_matrix)
```

Out[31]:

<matplotlib.axes._subplots.AxesSubplot at 0x28831710f88>



In [32]:

```
X=admission_df.drop(columns=['Chance of Admit'])
```

In [33]:

```
y=admission_df['Chance of Admit']
```

In [34]:

```
X.shape
```

Out[34]:

```
(500, 7)
```

In [35]:

```
y.shape
```

Out[35]:

```
(500,)
```

In [36]:

```
y
```

Out[36]:

```
0      0.92
1      0.76
2      0.72
3      0.80
4      0.65
```

```
...
495    0.87
496    0.96
497    0.93
498    0.73
499    0.84
```

Name: Chance of Admit, Length: 500, dtype: float64

In [37]:

```
X=np.array(X)
y=np.array(y)
```

In [38]:

```
y=y.reshape(-1,1)
y.shape
```

Out[38]:

```
(500, 1)
```

In [39]:

```
# scaling the data before training the model
from sklearn.preprocessing import StandardScaler, MinMaxScaler
#removes the mean and scales each feature/variable to unit variance. This operation is performed
scaler_x=StandardScaler() #MinMaxScaler
X=scaler_x.fit_transform(X)
print(X.mean(axis = 0))
print(X.std(axis = 0))
```

```
[ 1.76214598e-15  1.13686838e-15  1.42108547e-16 -8.52651283e-17
 4.26325641e-17  3.11928261e-15 -7.81597009e-17]
[1. 1. 1. 1. 1. 1.]
```

In [40]:

```
scaler_y=StandardScaler() #MinMaxScaler
y=scaler_y.fit_transform(y)
print(y.mean(axis = 0))
print(y.std(axis = 0))
```

```
[-3.30402372e-16]
[1.]
```

In [41]:

```
# splitting the data in to test and train sets
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.15)
```

In [42]:

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,accuracy_score
```

In [43]:

```
LinearRegression_model=LinearRegression()
LinearRegression_model.fit(X_train,y_train)
```

Out[43]:

```
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)
```

In [44]:

```
accuracy_LinearRegression=LinearRegression_model.score(X_test,y_test)
accuracy_LinearRegression
```

Out[44]:

```
0.6947776735807303
```

In [45]:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.python.keras.layers import Dense, Activation, Dropout
from tensorflow.python.keras.optimizers import Adam
# ** For tensorflow 2.0** #
# from tensorflow.keras.layers import Dense, Activation, Dropout
# from tensorflow.keras.optimizers import Adam
```

C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorflow\python\framework\dtypes.py:516: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (type, (1,)) / '(1,)type'.

```
_np_qint8 = np.dtype [("qint8", np.int8, 1)]
```

C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorflow\python\framework\dtypes.py:517: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (type, (1,)) / '(1,)type'.

```
_np_quint8 = np.dtype [("quint8", np.uint8, 1)]
```

C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorflow\python\framework\dtypes.py:518: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (type, (1,)) / '(1,)type'.

```
_np_qint16 = np.dtype [("qint16", np.int16, 1)]
```

C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorflow\python\framework\dtypes.py:519: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (type, (1,)) / '(1,)type'.

```
_np_quint16 = np.dtype [("quint16", np.uint16, 1)]
```

C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorflow\python\framework\dtypes.py:520: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (type, (1,)) / '(1,)type'.

```
_np_qint32 = np.dtype [("qint32", np.int32, 1)]
```

C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorflow\python\framework\dtypes.py:525: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (type, (1,)) / '(1,)type'.

```
np_resource = np.dtype [("resource", np.ubyte, 1)]
```

C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:541: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (type, (1,)) / '(1,)type'.

```
_np_qint8 = np.dtype [("qint8", np.int8, 1)]
```

C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:542: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (type, (1,)) / '(1,)type'.

```
_np_quint8 = np.dtype [("quint8", np.uint8, 1)]
```

C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:543: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (type, (1,)) / '(1,)type'.

```
_np_qint16 = np.dtype [("qint16", np.int16, 1)]
```

C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:544: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as (type, (1,)) / '(1,)type'.

```
_np_quint16 = np.dtype(["quint16", np.uint16, 1])
C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorboard\compat\tensorflow_
stub\dtypes.py:545: FutureWarning: Passing (type, 1) or '1type' as a synon
ym of type is deprecated; in a future version of numpy, it will be underst
ood as (type, (1,)) / '(1,)type'.
_np_qint32 = np.dtype(["qint32", np.int32, 1])
C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorboard\compat\tensorflow_
stub\dtypes.py:550: FutureWarning: Passing (type, 1) or '1type' as a synon
ym of type is deprecated; in a future version of numpy, it will be underst
ood as (type, (1,)) / '(1,)type'.
np_resource = np.dtype(["resource", np.ubyte, 1])
```

In [46]:

```

ANN_model = keras.Sequential()

ANN_model.add(Dense(50, input_dim = 7))
ANN_model.add(Activation('relu'))

ANN_model.add(Dense(150))
ANN_model.add(Activation('relu'))
ANN_model.add(Dropout(0.5))

ANN_model.add(Dense(150))
ANN_model.add(Activation('relu'))
ANN_model.add(Dropout(0.5))

ANN_model.add(Dense(50))
ANN_model.add(Activation('linear'))
ANN_model.add(Dense(1))

ANN_model.compile(loss = 'mse', optimizer = 'adam')
ANN_model.summary()

```

WARNING:tensorflow:From C:\Users\VATSAL\Anaconda3\lib\site-packages\tensorflow\python\ops\init_ops.py:1251: calling VarianceScaling.__init__ (from tensorflow.python.ops.init_ops) with dtype is deprecated and will be removed in a future version.

Instructions for updating:

Call initializer instance with the dtype argument instead of passing it to the constructor

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
dense (Dense)	(None, 50)	400
activation (Activation)	(None, 50)	0
dense_1 (Dense)	(None, 150)	7650
activation_1 (Activation)	(None, 150)	0
dropout (Dropout)	(None, 150)	0
dense_2 (Dense)	(None, 150)	22650
activation_2 (Activation)	(None, 150)	0
dropout_1 (Dropout)	(None, 150)	0
dense_3 (Dense)	(None, 50)	7550
activation_3 (Activation)	(None, 50)	0
dense_4 (Dense)	(None, 1)	51
=====		
Total params: 38,301		
Trainable params: 38,301		
Non-trainable params: 0		

In [47]:

```
ANN_model.compile(optimizer='Adam', loss='mean_squared_error')
```

In [48]:

```
epochs_hist = ANN_model.fit(X_train, y_train, epochs = 100, batch_size = 20, validation_split=0.2)
```

Train on 340 samples, validate on 85 samples

Epoch 1/100

340/340 [=====] - 2s 5ms/sample - loss: 0.6436 - val_loss: 0.2461

Epoch 2/100

340/340 [=====] - 0s 265us/sample - loss: 0.3567 - val_loss: 0.2015

Epoch 3/100

340/340 [=====] - 0s 331us/sample - loss: 0.3755 - val_loss: 0.2391

Epoch 4/100

340/340 [=====] - 0s 306us/sample - loss: 0.3016 - val_loss: 0.1807

Epoch 5/100

340/340 [=====] - 0s 358us/sample - loss: 0.2670 - val_loss: 0.1767

Epoch 6/100

340/340 [=====] - 0s 307us/sample - loss: 0.2899 - val_loss: 0.1764

Epoch 7/100

In [49]:

```
result = ANN_model.evaluate(X_test, y_test)
accuracy_ANN = 1 - result
print("Accuracy : {}".format(accuracy_ANN))
```

75/75 [=====] - 0s 119us/sample - loss: 0.2243

Accuracy : 0.7756882747014364

In [50]:

```
epochs_hist.history.keys()
```

Out[50]:

```
dict_keys(['loss', 'val_loss'])
```

In [51]:

```
plt.plot(epochs_hist.history['loss'])  
plt.title('Model Loss Progress During Training')  
plt.xlabel('Epoch')  
plt.ylabel('Training Loss')  
plt.legend(['Training Loss'])
```

Out[51]:

<matplotlib.legend.Legend at 0x28839583808>



Task 8 (Graduate_admission_Prediction.ipynb)

In [52]:

```

from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
y_pred = ANN_model.predict(X_test)
MSE = mean_squared_error(y_pred, y_test)
MAE = mean_absolute_error(y_pred, y_test)
RMSE = mean_squared_error(y_pred, y_test, squared=False)
r2 = r2_score(y_test, y_pred)
n = len(admission_df)
p = 7
adj_r2 = 1-(1-r2)*(n-1)/(n-p-1)
print('RMSE =',RMSE, '\nMSE =',MSE, '\nMAE =',MAE, '\nR2 =', r2, '\nAdjusted R2 =', adj_r2)

```

```

RMSE = 0.4736155873917128
MSE = 0.2243117246203971
MAE = 0.3168678284919182
R2 = 0.7271463871625696
Adjusted R2 = 0.7232643235652891

```

Task 10 (Graduate_admission_Prediction.ipynb)

In [53]:

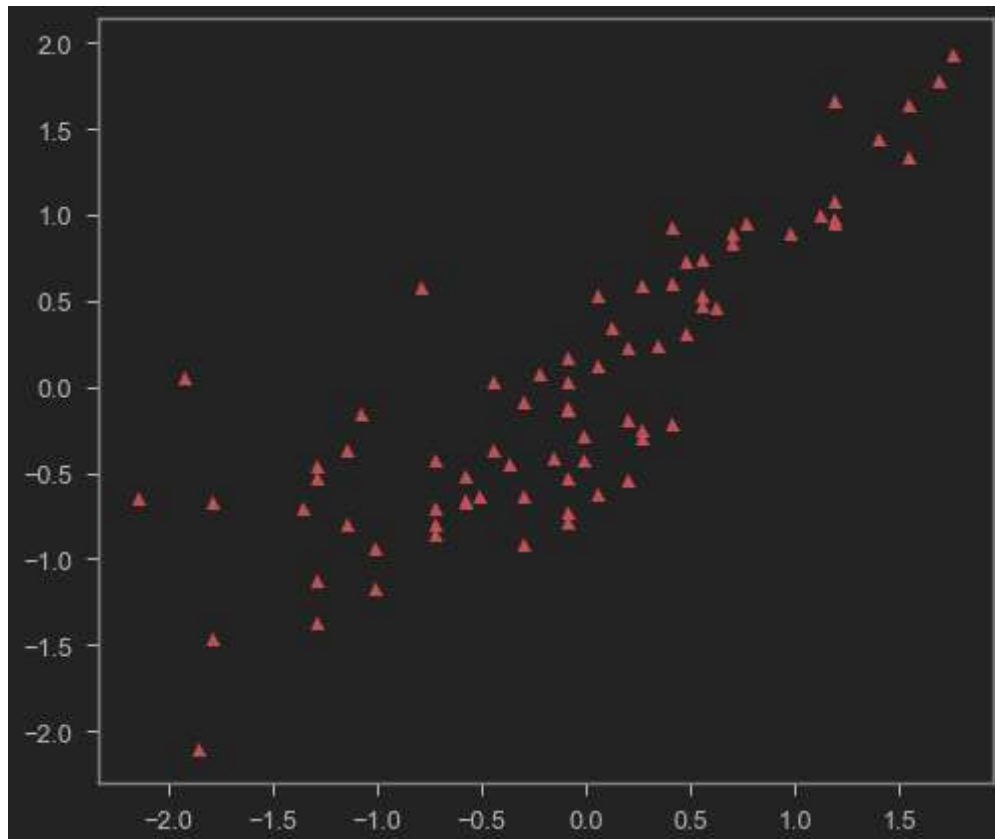
```

y_predict=LinearRegression_model.predict(X_test)
plt.plot(y_test,y_predict,'^', color='r')

```

Out[53]:

```
[<matplotlib.lines.Line2D at 0x28839727d48>]
```



In [54]:

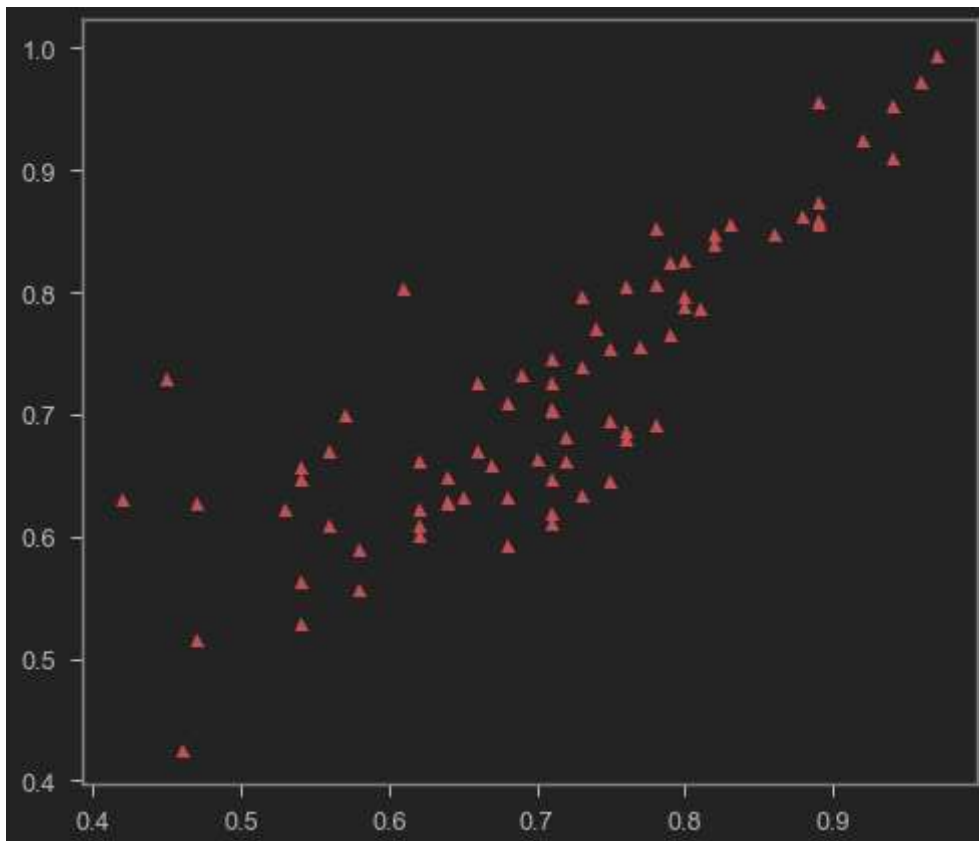
```
y_predict_original=scaler_y.inverse_transform(y_predict)
y_test_original=scaler_y.inverse_transform(y_test)
```

In [55]:

```
plt.plot(y_test_original,y_predict_original,'^',color='r')
```

Out[55]:

[<matplotlib.lines.Line2D at 0x2883962ffc8>]



In [56]:

```
k = X_test.shape[1]
n = len(X_test)
n
```

Out[56]:

75

In [57]:

```
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from math import sqrt
# START CODE#
RMSE = mean_squared_error(y_predict, y_test, squared=False)
MSE = mean_squared_error(y_predict, y_test)
MAE = mean_absolute_error(y_predict, y_test)
r2 = r2_score(y_test, y_predict)
adj_r2 = 1-(1-r2)*(n-1)/(n-k-1)

# END CODE#

print('RMSE =',RMSE, '\nMSE =',MSE, '\nMAE =',MAE, '\nR2 =', r2, '\nAdjusted R2 =', adj_r2)
```

```
RMSE = 0.5009210497495679
MSE = 0.250921898082209
MAE = 0.3482956713636489
R2 = 0.6947776735807303
Adjusted R2 = 0.6628887738055826
```